



FIREBRAND

LYCEUM EDUCATION GROUP

Version Control and Collaboration

Learning to save, track and share your code the way professional teams do.

What We're Covering Today

By the end of today you will save, track and share code the way real teams do.

1

Git fundamentals: initialise a project, stage and commit changes, understand what each command actually does.

2

Reading history: view a project's commit history and understand it as a record of decisions, not just files.

3

Branching: understand why teams work on separate branches instead of one shared file.

4

Remotes and sharing: connect a local project to a remote so others can see and use your work.

5

Team habits: commit messages, commit frequency, and communication as professional skills, not just commands.

The Problem With final-version-2-REALLY-FINAL.js

“

If your folder has a file called final-final-v3-USE-THIS.js, you already need version control.

A familiar developer joke, and a familiar developer truth

What Is Version Control, Really

Version control is a system that records changes to files over time.

1

A safety net: every saved change is recoverable, so nothing is ever truly lost.

2

A history: you can see exactly what changed, when, and read a note explaining why.

3

A shared workspace: multiple people can work on the same codebase without overwriting each other.

4

A professional norm: nearly every employer, in every sector, expects it as baseline practice.

Git vs GitHub, Cleared Up

Git

- A tool installed on your computer
- Tracks changes to files locally
- Works completely offline

GitHub

- A website that hosts Git projects online
- Provides a shared remote copy
- Adds collaboration features like pull requests

The Save-Track-Share Cycle

Git habits repeat in a loop, over and over, on every single project.

- 1 **Make a change to a file**
- 2 **Stage the change you want to keep**
- 3 **Commit it with a message explaining why**
- 4 **Repeat as work continues**

Starting a Project With Git

```
$ $ cd learner-profile-project
```

```
$ git init
```

```
Initialized empty Git repository in /learner-profile-project/.git/
```

First Git Setup

Create a folder, add a file, and turn it into a Git repository.

- 1 Create a new folder called git-practice on your desktop.
- 2 Copy your Day 1 my-learner-profile.js file into it.
- 3 Open a terminal in that folder and run git init.
- 4 Confirm you see the "Initialized empty Git repository" message.



TIMEBOX

10 minutes

**Individual work, checking
with a neighbour if stuck**



OUTPUT

**A folder that is now a working Git
repository containing one
JavaScript file.**

Git's Three Areas

Git organises your work into three distinct areas, each with a specific job.

1

Working directory

the actual files you're editing, exactly as you see them in your editor.

2

Staging area

a holding area for changes you've chosen to include in the next save.

3

Repository history

the permanent record of every commit you've made so far.

Staging and Committing

```
$ $ git add my-learner-profile.js
$ git commit -m "Add initial learner profile script"
[main (root-commit) a1b2c3d] Add initial learner profile script
1 file changed, 12 insertions(+)
```

Your First Commit

Stage and commit your learner profile file with a clear message.

- 1 Run `git add my-learner-profile.js` in your `git-practice` folder.
- 2 Run `git commit -m` with a short, honest message describing the file.
- 3 Read the confirmation output Git gives you.
- 4 Compare your commit message with a neighbour's.



TIMEBOX

10 minutes



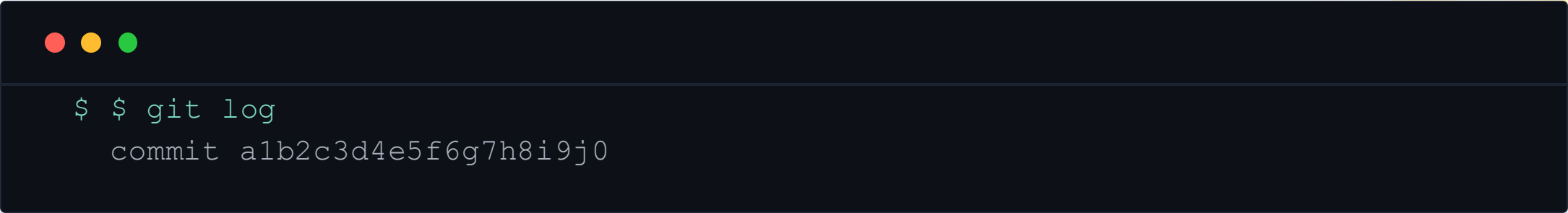
OUTPUT

A first commit recorded in your project's history, with a message you chose.

A Good Commit Message Does This

- 1 It describes the change, not the file.
Say what changed in behaviour or logic, not just which file was touched.
- 2 It explains why, when the why isn't obvious.
A reader should understand the reasoning, not just the mechanics.
- 3 It stays short and specific.
One clear sentence beats a vague paragraph every time.

Viewing Your History



```
$ $ git log  
commit a1b2c3d4e5f6g7h8i9j0
```

Check Your Own History

Run git log on your project and read your own commit back.

- 1 Run git log in your git-practice folder.
- 2 Find your commit message in the output.
- 3 Identify the author name and date shown.
- 4 Discuss with a neighbour what each part of the output means.



TIMEBOX

8 minutes



OUTPUT

**A correctly read commit entry,
with each part of the output
identified.**

One Commit Is a Photo. History Is a Film.

“

A single commit is a snapshot. A string of commits is the story of how the code got here.

A useful way to think about Git history

Making a Second Change

Comparing the file before and after a small edit, ready to stage and commit again

my-learner-profile.js (before)

```
1 let learnerName = "Sam";  
2 let learnerCourse = "Level 3 Diploma";  
3 console.log(learnerName + " is studying "
```

my-learner-profile.js (after)

```
1 let learnerName = "Sam";  
2 let learnerCourse = "Level 4 Software Development";  
3 console.log(learnerName + " is studying "
```


Commit a Second Change

Edit your file, then stage and commit the change with a new message.

- 1 Open my-learner-profile.js and change the course name value.
- 2 Save the file.
- 3 Run git add and then git commit -m with a new message.
- 4 Run git log to see both commits now listed.



TIMEBOX

10 minutes



OUTPUT

Two commits visible in your project's history, each with its own message.

Morning So Far

A quick check of the ground covered before we build on it further.

1

Git tracks changes to a project over time, recoverable and explainable.

2

git init starts tracking a folder, once, at the start of a project.

3

git add stages a change, git commit -m saves it permanently with a message.

4

git log shows the full history of commits, each with author, date and message.

What If Two People Edit the Same File?

A retail platform team is adding two new features to the same checkout script.

1

CONTEXT

Priya is adding a discount code field. Tunde is adding a delivery date picker, in the same file, at the same time.

2

CHALLENGE

If they both edit the same file directly and save over each other, one person's work disappears without warning.

3

TASK

Consider what a version control system could offer here that a single shared file cannot.

Branches: Working Without Collision

A branch is an independent line of work that splits off from the main project.

Main branch

the stable, working version of the project that everyone shares.

Feature branch

a copy of main where one person or pair can work safely.

Independent changes

each branch can be edited without affecting the other.

Merging

bringing a finished branch's changes back into main once ready.

Branches Are Not Just for Big Teams

Branching is a useful habit for a team of two just as much as a team of two hundred.

- 1 Safety: try an idea on a branch without risking the version that already works.
- 2 Focus: one branch per feature or fix keeps changes easy to review and understand.
- 3 Freedom to experiment: a branch that doesn't work out can simply be abandoned.
- 4 A universal habit: used the same way whether the team is a solo developer or a large one.

Creating and Switching Branches

```
$ $ git branch feature-update  
$ git switch feature-update  
Switched to branch 'feature-update'
```

Create Your Own Branch

Create a new branch and switch to it in your git-practice project.

- 1 Run git branch add-greeting in your project folder.
- 2 Run git switch add-greeting to move onto it.
- 3 Confirm the terminal shows you're now on that branch.



TIMEBOX

8 minutes



OUTPUT

A new branch created and switched to, ready for its own changes.

A Change on the New Branch

Adding a friendly greeting line to the profile script, but only on this branch.

my-learner-profile.js

```
1 let learnerName = "Sam";  
2 let learnerCourse = "Level 4 Software Developer Apprenticeship";  
3 console.log("Welcome back, " + learnerName + "!");  
4 console.log(learnerName + " is studying " + learnerCourse);
```


Commit on Your Branch

Add the greeting line, then stage and commit it on your new branch.

- 1 Add the extra console.log greeting line shown on the previous slide.
- 2 Save the file.
- 3 Stage and commit the change with a clear message.
- 4 Run git log to confirm the commit is recorded.



TIMEBOX

10 minutes



OUTPUT

**A commit that exists only
on the add-greeting
branch.**

Branches in One Sentence



A branch lets you make changes safely, without touching the version everyone else relies on.

Morning Recap

- What does staging actually do, in your own words?
- Why does a commit message matter to someone reading it in six months?
- What problem does a branch solve that a single shared file cannot?

Discuss your answers with a neighbour before we break.



FIREBRAND
LYCEUM EDUCATION GROUP

Lunch

An hour to step away before this afternoon's remotes, sharing and team habits.

Welcome Back

This afternoon moves from your own machine to sharing work with others.

1

Remotes: connecting your local project to a shared online copy.

2

Push and pull: sending your commits up, and bringing others' commits down.

3

Cloning: getting a copy of an existing project to work on.

4

Team habits: how good Git practice supports real collaboration, not just solo backups.

A Number Worth Sitting With

“

Git remains the version control system used by the overwhelming majority of professional software teams worldwide.

Stack Overflow Developer Survey, most recent editions

What Is a Remote?

A remote is a version of your project stored somewhere other than your own machine.

1

A shared copy: everyone on the team can access the same remote project.

2

A backup: your work exists somewhere beyond your own laptop.

3

A sync point: pushing sends your commits there, pulling brings others' commits to you.

4

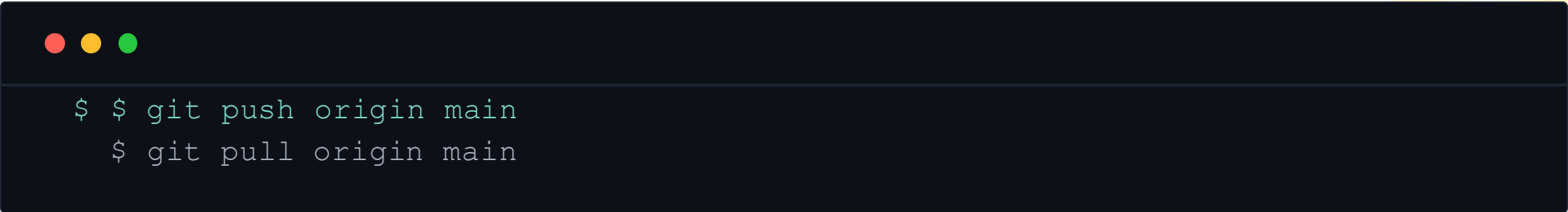
Not a replacement: your local repository still holds the full history too.

How Push and Pull Work

Commits travel between your machine and the remote in two directions.

- 1** **YOU COMMIT LOCALLY**
your change is saved to your own history first.
- 2** **YOU PUSH**
your commit travels up to the shared remote.
- 3** **A TEAMMATE PULLS**
they bring your commit down into their own local copy.
- 4** **THEY COMMIT AND PUSH**
the cycle continues in both directions.

Push and Pull Commands



```
$ $ git push origin main  
$ git pull origin main
```

Connecting to a Remote

```
$ $ git remote add origin https://example.com/git-practice.git
$ git remote -v
origin https://example.com/git-practice.git (fetch)
origin https://example.com/git-practice.git (push)
```

Connect and Confirm

Connect your project to a remote and confirm the connection.

- 1 Run git remote add origin followed by your provided remote address.
- 2 Run git remote -v to confirm the connection.
- 3 Check that both fetch and push lines show your address.
- 4 Ask a neighbour to check their output looks the same shape as yours.



TIMEBOX

10 minutes



OUTPUT

**A local project successfully
connected to a remote
repository.**

Pushing Your Work

Push your committed work up to the remote you just connected.

- 1 Run git push origin main in your git-practice project.
- 2 Read the confirmation output in your terminal.
- 3 Check your remote's website or dashboard to see the pushed commits.
- 4 Note anything unexpected to discuss as a group.



TIMEBOX

12 minutes



OUTPUT

**Your local commits now
visible on a shared remote
copy.**

The Other Direction: Cloning

Cloning downloads an entire existing remote project, history included, onto your machine.

1

A full copy: every commit, every message, every branch comes with it.

2

The usual starting point: joining an existing team project almost always starts with a clone.

3

One command: `git clone` followed by the project's address.

4

Nothing to set up first: cloning creates the folder and the Git connection for you.

Cloning a Project

```
$ $ git clone https://example.com/team-checkout-project.git
Cloning into 'team-checkout-project'...
remote: Enumerating objects: 42, done.
Receiving objects: 100% (42/42), done.
```

Try a Clone

Clone a shared practice project and explore its existing history.

- 1 Run git clone with the shared project address provided.
- 2 Move into the new folder that appears.
- 3 Run git log to read through its existing commit history.
- 4 Identify one commit message you think is genuinely clear.



TIMEBOX

12 minutes



OUTPUT

A cloned project on your machine, with its history explored and understood.

It's Not Really About the Commands



Git gives teams a shared, honest record of what happened and why.

Habits That Make Git Work for a Team

The committer

- Write messages a stranger could understand
- Commit small, focused changes rather than huge ones
- Pull before starting new work each day

The reviewer

- Read commit history before asking "what changed"
- Check the reasoning in the message, not just the code
- Ask questions rather than assuming intent

The team

- Agree a shared naming style for branches
- Communicate before working on the same file
- Treat the remote as the single source of truth

Spot the Weak Commit Message

```
$ $ git log --oneline
a3f9c21 fix
b7e2d10 more changes
c1a8f44 stuff
d9b3e77 Add empty-input guard to access checker
e5c6f21 update
```

Rewrite the Weak Messages

Rewrite three vague commit messages from the previous slide into clear ones.

- 1 Choose three of the four vague messages shown on the previous slide.
- 2 Invent a plausible real change each message might describe.
- 3 Rewrite each as a clear, specific commit message.
- 4 Compare your rewrites with a neighbour's.



TIMEBOX

10 minutes



OUTPUT

Three clear, specific commit messages rewritten from vague originals.

The Afternoon Build: Bringing It All Together

You're picking up a shared project and contributing a small, well-documented change.

1

CONTEXT

A logistics retailer's small team maintains a delivery status checker script, similar to work from earlier in this programme.

2

CHALLENGE

You need to add a new feature safely, on your own branch, with clear commits, ready to share back to the team.

3

TASK

Clone the shared project, branch, make a change, commit it clearly, and push it back.

Starter File for the Build

The shared delivery status checker script learners will branch from and extend.

```
delivery-status.js

1 let deliveryDays = 3;
2 let isFragile = true;
3 if (deliveryDays <= 2) {
4   console.log("Status: Priority delivery");
5 } else if (deliveryDays <= 5) {
6   console.log("Status: Standard delivery");
7 } else {
8   console.log("Status: Delayed delivery");
9 }
```

Build Steps, Part One

Clone the shared project and create your own branch to work on.

- 1 Clone the shared delivery-status project using the address provided.
- 2 Move into the new project folder.
- 3 Run git log to review its existing history.
- 4 Create and switch to a branch called fragile-warning.



TIMEBOX

10 minutes



OUTPUT

A cloned project with your own branch created and active.

The Feature to Add

Adding a fragile-item warning line using the existing isFragile variable

delivery-status.js (before)

```
1 if (deliveryDays <= 2) {  
2   console.log("Status: Priority delivery");  
3 } else if (deliveryDays <= 5) {  
4   console.log("Status: Standard delivery");  
}
```

delivery-status.js (after)

```
1 if (deliveryDays <= 2) {  
2   console.log("Status: Priority delivery");  
3 } else if (deliveryDays <= 5) {  
4   console.log("Status: Standard delivery");  
5   if (isFragile) {  
6     console.log("Note: Handle with care, fragile item");  
7   }  
}
```

Build Steps, Part Two

Add the fragile warning, then commit and push it on your branch.

- 1 Add the fragile-item warning shown on the previous slide.
- 2 Save the file and test it by running the script.
- 3 Stage and commit the change with a clear message.
- 4 Push your branch to the remote using git push origin fragile-warning.



TIMEBOX

18 minutes



OUTPUT

A pushed branch containing one clearly committed feature, ready to share.

What Comes After a Push

Once work is pushed, most real teams follow a similar next step.

1

PUSH THE BRANCH

your change is now visible on the shared remote.

2

OPEN A REQUEST TO MERGE

a teammate is asked to review it.

3

A COLLEAGUE REVIEWS

they check the code and the reasoning behind it.

4

MERGE INTO MAIN

once approved, the change joins the main project.

A Full Circle



Today you did, in miniature, exactly what a professional developer does every working day.

Two Ways of Working, Compared

Working without version control

- Files renamed manually to track versions
- Changes overwritten with no history
- Sharing means emailing a file or copying a folder
- No record of who changed what, or why

Working with Git

- Every change recorded with a message explaining it
- Nothing is ever truly lost, history is always recoverable
- Sharing happens through push and pull on a remote
- A clear, searchable record of who changed what, and why

Consolidation: Explain the Loop

Explain the full clone-branch-edit-commit-push loop to a partner, in your own words.

- 1 Without looking at slides, describe each step in the loop aloud.
- 2 Use today's delivery-status build as your example.
- 3 Ask your partner to spot any step you missed or blurred.
- 4 Swap roles and listen to their explanation in turn.



TIMEBOX

10 minutes



OUTPUT

A clear spoken explanation of the full workflow, checked by a partner.

Consolidation: Spot the Missing Step

Identify what's missing from a flawed description of the Git workflow.

- 1 Read this claim: "I edited the file, then pushed it straight away."
- 2 Identify which step or steps are missing from that description.
- 3 Explain what would actually happen if someone tried this.
- 4 Compare your answer with a neighbour's.



TIMEBOX

8 minutes



OUTPUT

A correct identification of the missing staging and commit steps.

Exit Ticket

Four short written reflections to close out today's learning.

- 1 Write one sentence explaining what staging means, in your own words.
- 2 Name one reason a clear commit message matters to a team.
- 3 Explain why branches let two people work without colliding.
- 4 Note one thing you want more practice with before next session.



TIMEBOX

12 minutes



OUTPUT

A four-part written reflection capturing today's key ideas.

Closing

Today you learned to save, track and share code the way professional teams genuinely work.



Git records every change with a message, building an honest, searchable history over time.



Branches let you work safely on your own line of change, without disturbing what already works.



Remotes turn a private habit into a shared one, through pushing and pulling commits.



None of this replaces good communication, it supports it.